

Assignment 2-Case Study

Stage 3 Tutorial Activities

From Requirements to Domain Models

Week 6 | 60 minutes

Candidate Concepts

Candidate	Class?	Reason
Patient	Yes	It's a main person we need to store information about for example, their contact details.
Practitioner	Yes	We need to track the staff members and their daily schedules.
Appointment	Yes	This is the main event that connects a patient and a practitioner together at a certain time.
Name	No	This is just a piece of data that belongs to a patient or practitioner, not a whole class on its own.
Clinic	No	Since we are only building this for one single clinic, we don't need a class for it.
Database	No	This is a technological detail about how data is saved, not a real-world concept for our models
Cancellation	No	This is just a status tag we can put on an appointment; it doesn't need to be its own thing.
Status	No	Just like cancellation, this is just a filed inside the appointment class.

CRC Cards

Patient

Responsibilities	Collaborators
Knows past appointment history	Appointment
Knows personal details (ID, Full Name, Phone number, etc.)	Appointment

Practitioner

Responsibilities	Collaborators
Know practitioner details	Appointment
Know assigned daily schedule	Appointment

Appointment

Responsibilities	Collaborators
Know scheduled date, time and status	Appointment
Link a specific patient to a practitioner	Patient and practitioner

Relationship Reasoning

Patient to Appointment: which relationship and why?

A one-to-many relationship, a patient can have zero or multiple appointments over time.

Practitioner to Appointment: what multiplicity?

A one-to-many relationship, a practitioner handles multiple appointments.

Should Appointment inherit from patient?

No, inheritance represents an “is-a” relationship. This should be an association for example, “has-a.”

Does Clinic need to own every object?

No, creating a central class that owns everything makes the system rigid and hard to maintain.

AI Model Critique

Critique AI proposals: PatientManager, PractitionerManager, AppointmentManager, ClinicController, NotificationManager, ScheduleEngine.

The AI suggested classes like PatientManager, ClinicController, and ScheduleEngine, but these should be rejected. A domain model is only supposed to map out real-world concepts from the actual business, like a Patient or an Appointment. The AI's suggestions are software implementation details (how the actual code is organized), not real-world domain concepts, so they do not belong in this model.